

DATAPLANE ROUTER

Prezentare generală

În cadrul temei vom implementa dataplane-ul unui router. Un router are două părți:

- Dataplane - partea care implementează procesul de dirijare propriu-zis, pe baza intrărilor din tabela de rutare.
- Control plane - partea care implementează algoritmi de rutare (e.g. RIP, OSPF, BGP); acești algoritmi distribuți calculează intrările din tabela de rutare.

Tema constă în implementarea componentei de dataplane. În cele ce urmează, în lipsa altor precizări, toate referințele la router din textul temei se referă la dataplane.

NU va fi nevoie să implementați acești algoritmi în cadrul temei; vom lucra strict din perspectiva dataplane-ului, pentru care tabelele de rutare sunt deja populate. Routerul nostru va funcționa cu o tabelă de rutare statică, primită într-un fișier de intrare, și care nu se va schimba pe parcursul rulării.

Un router are mai multe interfețe și poate recepționa pachete pe oricare dintre acestea. Routerul trebuie să transmită pachetul mai departe, către un calculator sau către alt router direct conectat, în funcție de regulile din tabela de rutare.

Tema poate fi implementată în C sau C++.

Lectură tema

Tema presupune cunoștințele dobândite la curs. Vă recomandăm parcurgerea următoarei bibliografii înainte de a vă apuca de temă.

- [The Internet Protocol \(9 min\)](#)
- [ARP: Mapping between IP and Ethernet \(10 min\)](#)
- [Looking at ARP and ping packets \(8 min\)](#)
- [Hop-by-hop routing \(13 min\)](#)
- Capitolul 4.3 (What's Inside a Router) din [Computer Networking: A Top-Down Approach \(6th Edition\)](#)
- Capitolul 4.4.3 (Internet Control Message Protocol (ICMP)) din [Computer Networking: A Top-Down Approach \(6th Edition\)](#)
- Capitolul 5.4.1 (Link-Layer Addressing and ARP) din [Computer Networking: A Top-Down Approach \(6th Edition\)](#)

Deadline

Deadline-ul temei este specificat pe moodle. Conform regulamentului:

- După expirarea acestui termen limită se mai pot trimite teme un interval de maxim 3 zile, cu următoarele depuneri: 10p în prima zi, 20p în a doua zi și 30p în a treia zi.
- După cele 3 zile tema nu se mai poate trimite.
- Oferim posibilitatea fiecărui student de a avea un număr de maxim **5 zile** numite "sleep days".
- Aceste zile pot fi folosite pentru a amâna termenul de predare al temei de casă (fără penalizări).
- Nu se pot folosi mai mult de **două** sleep days pentru o temă de casă.
- Pentru a utiliza aceste zile completați formularul de pe Moodle
- Temele de casă sunt *individuale*.

Setup

Pentru a simula o rețea virtuală vom folosi [Mininet](#). Mininet este un simulator de rețele ce folosește în simulare implementari reale de kernel, switch și cod de aplicații.

```
sudo apt update
```

```
sudo apt install mininet openvswitch-testcontroller tshark python3-click python3-scapy xterm
```

Verificați să aveți instalat tshark.

După ce am instalat Mininet, vom folosi următoarea comandă pentru a crește dimensiunea fontului în terminalele pe care le vom deschide.

```
echo "xterm*font: *-fixed-*,*-18-*" >> ~/.Xresources
```

```
xrdb -merge ~/.Xresources
```

Dacă nu merge așa, o altă opțiune este să modificați .Xresources cu:

```
xterm*faceName: Monospace
```

```
xterm*faceSize: 14
```

Și să rulați `xrdb -merge ~/.Xresources`.

Procesul de dirijare (forwarding)

Dirijarea este un proces care are loc la nivelul 3 ("Rețea") din stiva OSI. În momentul în care un pachet ajunge la router, acesta trebuie să efectueze următoarele acțiuni:

1. **Parsarea pachetului:** din fluxul de octeți ce reprezintă pachetul, routerul trebuie să afle ce antete sunt prezente și ce valori conțin câmpurile acestora, construind și inițializând structuri interne corespunzătoare. La acest pas, dacă routerul descoperă că un pachet primit e malformat (e.g. prea scurt), îl aruncă.
2. **Validarea L2:** pachetele conțin un antet de Ethernet; routerul nostru trebuie să considere doar pachetele trimise către el însuși (i.e. câmpul MAC destination este același cu adresa MAC a interfeței pe care a fost primit pachetul) sau către toată lumea (i.e. câmpul MAC destination este adresa de *broadcast*, FF:FF:FF:FF:FF:FF). Orice alt pachet trebuie aruncat.
3. **Inspectarea următorului header:** routerul inspectează câmpul Ether Type din header-ul Ethernet pentru a descoperi următorul antet prezent. În cadrul temei, ne vor interesa doar două posibilități: [IPv4](#) și [ARP](#). Următoarele acțiuni ale routerului vor depinde de tipul acestui header. Orice alt fel de pachet trebuie ignorat.

În continuare, vom studia protocoalele implicate în procesul de dirijare și vom prezenta cum acestea sunt folosite de către router.

Ethernet

Ethernet este echivalentul protocolului de DataLink pe care l-am implementat în primele laboratoare. Noi vom lucra doar cu **cadre Ethernet** ce sunt transmise ca payload peste implementarea protocolului de nivel fizic Ethernet. Cum CRC-ul este calculat în hardware, nu o să îl regăsim în header. În acest caz, header-ul pe care îl vom folosi este următorul:

```
Ethernet L2 Frame Header
+-----+-----+-----+
|   Bytes 0-5   | Bytes 6-11 | Bytes 12-13 |
+-----+-----+-----+
| Destination MAC | Source MAC | EtherType   |
+-----+-----+-----+
```

Adresa MAC Destinație reprezintă identificatorul dispozitivului de nivel 2 către care a fost trimis acest cadru.

În cadrul temei vom folosi următoarea structură pentru un cadru Ethernet.

```
// include/protocols.h
```

```
struct ether_hdr {
```

```
    uint8_t ether_dhost[6]; //adresa mac destinatie
```

```
    uint8_t ether_shost[6]; //adresa mac sursa
```

```
    uint16_t ether_type;    // identicator protocol encapsulat
```

```
};
```

În [RFC 5342](#) sunt definite valorile pe care ethertype le poate lua. În cazul nostru, ne interesează IPv4 (0x0800) și ARP (0x0806).

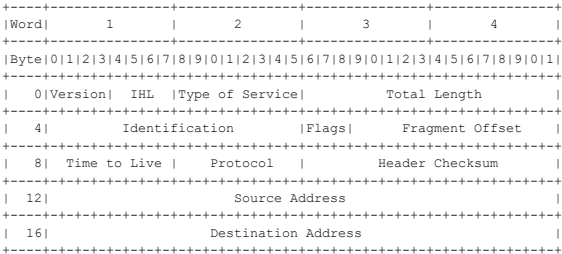
Noi vom folosi API-ul de nivel 2 pentru a trimite cadre Ethernet L2 peste protocolul de L1 Ethernet ce se ocupa cu framing, checksums etc. [Aici](#) găsiți o descriere completă a structurii Ethernet. În payload-ul cadrului de Ethernet vom avea encapsulat fie protocolul ARP, fie protocolul IP.

În figura de mai jos, găsim o captură Wireshark ce surprinde un cadru ethernet de L2 în care avem encapsulat IP și ICMP peste IP.

```
> Frame 130: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface enp37s0, id 0
> Ethernet II, Src: ASRockIn_7d:ee:86 (70:85:c2:7d:ee:86), Dst: Microsof_14:02:63 (00:15:5d:14:02:63)
  > Destination: Microsof_14:02:63 (00:15:5d:14:02:63)
  > Source: ASRockIn_7d:ee:86 (70:85:c2:7d:ee:86)
    Type: IPv4 (0x0800)
  > Internet Protocol Version 4, Src: 172.19.2.160, Dst: 142.250.186.78
  > Internet Control Message Protocol
```

Protocolul IPv4

Protocolul IP este utilizat pentru a permite dispozitivelor conectate în rețele diferite să schimbe informații prin intermediul unui dispozitiv intermediar numit router. O descriere completă a header-ului de IPv4 o găsiți [aici](#). Header-ul unui pachet (packet) IP este următorul:



Următoarea structură poate fi folosită pentru a reprezenta un pachet IPV4.

```
// include/protocols.h
struct ip_hdr {
    // this means that version uses 4 bits, and ihl 4 bits
    uint8_t ihl:4, ver:4; // we use version = 4
    uint8_t tos; // Nu este relevant pentru temă (set pe 0)
    uint16_t tot_len; // total length = ipheader + data
    uint16_t id; // Nu este relevant pentru temă, (set pe 4)
    uint16_t frag; // Nu este relevant pentru temă, (set pe 0)
    uint8_t ttl; // Time to Live -> to avoid loops, we will decrement
    uint8_t proto; // Identificator al protocolului encapsulat (e.g. ICMP)
    uint16_t checksum; // checksum -> Since we modify TTL,
    uint32_t source_addr; // Adresa IP sursă
    uint32_t dest_addr; // Adresa IP destinație
};
```

Nu suntem interesați de anumite funcționalități din IP precum fragmentarea. Astfel, în cazul în care construiți un pachet IP de la 0, vom seta **obligatoriu** următoarele:

```
tos = 0
frag_off = 0
version = 4
ihl = 5
id = 4
```

În [RFC 990 pagina 24](#) găsiți o listă de identificatori pentru diversele protocoale ce pot fi encapsulate în IP. Vom folosi aceste valori pentru a completa câmpul protocol.

Când un router primește un pachet de tip IPv4, trebuie să realizeze următoarele acțiuni:

- Verifică dacă el este destinația:** deși un intermediar, routerul este de asemenea o entitate cu interfețe de rețea și adrese IP asociate acestora, deci poate fi destinatarul unui pachet. În acest caz, routerul nu trebuie să trimită mai departe pachetul, ci să îi înțeleagă conținutul pentru a putea acționa în consecință. În cadrul acestei teme, routerul va răspunde doar la mesaje de tip **ICMP**.
- Verifică checksum:** routerul trebuie să recalculeze suma de control a pachetului, și să o compare cu cea primită în antetul de IP; dacă sumele diferă, pachetul a fost corupt și trebuie aruncat.
- Verificare și actualizare TTL:** pachetele cu câmpul TTL având valoarea 1 sau 0 trebuie aruncate. Routerul va trimite înapoi, către emițătorul pachetului un mesaj ICMP de tip "Time exceeded" (mai multe detalii în [secțiunea ICMP](#)). Altfel, câmpul TTL e decrementat.
- Căutare în tabela de rutare:** routerul caută adresa IP destinație a pachetului în tabela de rutare pentru a determina adresa următorului hop, precum și interfața pe care va trebui scos pachetul. În caz că nu găsește nimic, pachetul este aruncat. Routerul va trimite înapoi, către emițătorul pachetului un mesaj ICMP de tip "Destination unreachable" (mai multe detalii în [secțiunea ICMP](#)).
- Actualizare checksum:** routerul recalculează suma de control a pachetului (această trebuie recalculată din cauza schimbării câmpului TTL) și o surpascie în antetul IP al pachetului.
- Rescriere adrese L2:** pentru a forma un cadru corect care să fie transmis la următorul hop, routerul are nevoie să rescrie adresele de L2: adresa sursă va fi adresa interfeței routerului pe care pachetul e trimis mai departe, iar adresa destinație va fi adresa MAC a următorului hop. Pentru a determina adresa următorului hop, routerul folosește protocolul [ARP](#).
- Trimiterea noului pachet pe interfața corespunzătoare următorului hop.**

Tabela de rutare

Fiecare router dispune de o "tabelă de rutare" -- o structură pe baza căreia alege portul pe care să emită un pachet. În mod normal, aceste tabele sunt populate de către control plane, în urma rulării unui algoritm de rutare (e.g. OSPF); în cadrul temei, vom folosi o tabelă statică.

O intrare în tabel are patru coloane:

- prefix și mask: împreună aceste două câmpuri formează o adresă de rețea (e.g. 192.168.1.0/24)
- next hop: adresa IP a mașinii către care va fi trimis pachetul; dacă routerul nu e conectat direct la destinatar, aceasta va fi adresa unui router intermediar.
- interface: identificatorul interfeței pe care routerul va trebui să trimită pachetul respectiv către next hop.

Prefix	Next hop	Mask	Interface
192.168.0.0	192.168.0.2	255.255.255.0	0
192.168.1.0	192.168.1.2	255.255.255.0	1
192.168.2.0	192.168.2.2	255.255.255.0	2
192.168.3.0	192.168.3.2	255.255.255.0	3

În contextul temei, este suficient să considerați că numărul maxim de intrări din tabela de rutare este de 100 000 de intrări.

Longest Prefix Match

Pentru adresa IPv4 destinație din pachetul primit, routerul trebuie să caute intrările din tabela de rutare care descriu o rețea ce cuprinde adresa respectivă. Este posibil ca mai multe intrări să se potrivească; în acest caz, routerul trebuie s-o aleagă pe cea mai specifică, i.e. cea cu masca cea mai mare. Acest criteriu de căutare se numește "Longest Prefix Match" (LPM).

Programatic, routerul poate verifica apartenența unei adrese la o rețea, făcând operația de AND pe biți și verificând că este egală cu prefixul:

```
ip.destination & entry.mask == entry.prefix
```

Odată ce routerul găsește toate prefixele care se potrivesc adresei destinație din pachet, va alege intrarea din tabela de rutare corespunzătoare celui mai lung prefix (masca cea mai mare).

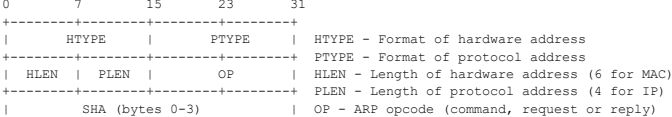
Protocolul ARP

După ce un router a determinat următorul hop pentru un pachet folosind LPM, trebuie să-l trimită mai departe, încapsulându-l într-un cadru cu adresa MAC destinație setată ca cea a următorului hop. Cum știe routerul această adresă? O opțiune ar fi ca această să fie reținută static, într-un tabel.

În realitate, însă, din mai multe motive (e.g. flexibilitate), routerul *nu știe* aceste adrese, ci trebuie să le determine folosind protocolul ARP ([RFC 826](#)). Având adresa IP a următorului hop precum și identificatorul interfeței care duce la acesta, routerul generează un mesaj de broadcast în rețeaua respectivă, întrebând care e adresa MAC asociată acelei adrese IP. Mașina respectivă observă că este vorba de propriul său IP și îi trimite routerului un mesaj cu propria adresă MAC. Acum routerul poate forma cadrul corespunzător; deasemena, păstrează într-un cache adresa MAC primită, pentru un interval limitat de timp.

Protocolul ARP este deci relevant în două puncte: atunci când primim un pachet cu antet ARP și atunci când trebuie să rescriem pentru forwardare un pachet IPv4.

Antetul protocolului ARP este următorul:



```

+-----+-----+-----+ SHA - Sender hardware address
| SHA (bytes 4-5) | SPA (bytes 0-1) | SPA - Sender IP address
+-----+-----+-----+
| SPA (bytes 2-3) | THA (bytes 0-1) | THA - Target hardware address
+-----+-----+-----+
| THA (bytes 2-3) | TPA (bytes 0-1) | TPA - Target IP address
+-----+-----+-----+
| THA (bytes 2-3) | THA (bytes 4-5) |
+-----+-----+-----+
| TPA (bytes 0-3) |
+-----+-----+-----+

```

Vom folosi urmatoarea structura pentru a reprezenta acest header în C:

```

// include/protocols.h
/* Ethernet ARP packet from RFC 826 */
struct arp_hdr {
    uint16_t hw_type; /* Format of hardware address */
    uint16_t proto_type; /* Format of protocol address */
    uint8_t hw_len; /* Length of hardware address */
    uint8_t proto_len; /* Length of protocol address */
    uint16_t opcode; /* ARP opcode (command) */
    uint8_t shwa[6]; /* Sender hardware address */
    uint32_t sprotoa; /* Sender IP address */
    uint8_t thwa[6]; /* Target hardware address */
    uint32_t tprotoa; /* Target IP address */
} __attribute__((packed));

```

Noi vom folosi ARP pentru a determina adresa MAC a unui host având adresa IPv4 a acestuia. Parametrii folosiți de ARP precum op sau htype îi găsiți [documentati de IANA](#). Cum facem translația din adrese IPv4, în ptype vom folosi identificatorul IPv4 conform [RFC 5342](#). hlen reprezintă dimensiunea adresei hardware, în acest caz adresa MAC are 6 bytes. plen reprezintă adresa protocolului de nivel network, în acest caz IPv4.

Exemplu de pachet de tip ARP request capturat cu Wireshark:

```

Ethernet II, Src: Micro-St_57:ff:f6 (d8:bb:c1:57:ff:f6), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: Micro-St_57:ff:f6 (d8:bb:c1:57:ff:f6)
  Sender IP address: 192.168.0.150
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 169.254.255.255

```

Pașii relevanți pentru dirijarea unui pachet IPv4 sunt următorii:

- Căutare în cache:** routerul se uită în cache-ul său ARP pentru a vedea dacă există o intrare curentă pentru adresa IPv4 a următorului hop. Dacă aceasta există, routerul o ia din cache, rescrie antetul L2 al pachetului și îl trimite mai departe.
- Salvare pachet pentru mai târziu:** dacă adresa necesară nu se găsește în cache-ul ARP, routerul va trebui să facă o interogare generând un pachet ARP și așteptând un răspuns. Pachetul original care trebuia dirijat este adăugat într-o coadă, pentru a putea fi trimis mai târziu, după sosirea răspunsului ARP.
- Generare ARP request:** routerul generează un pachet de tip ARP pentru a interoga despre adresa MAC a mașinii cu adresa IPv4 a următorului hop. Pentru asta are nevoie să genereze un pachet cu un antet Ethernet, urmat de un antet ARP.
 - Antetul Ethernet:** trebuie să conțină un ethertype care să identifice un pachet de tip ARP (0x806). Adresa MAC sursă va fi adresa interfeței routerului către next hop; adresa MAC destinație va fi cea de broadcast (FF:FF:FF:FF:FF:FF).
 - Antetul ARP:** trebuie să conțină tipul de adresă folosit în căutare (IPv4) împreună cu adresa în sine, precum și tipul de adresă căutată (MAC) împreună cu adresa în sine. Pentru mai multe detalii, consultați [\[@arprfc\]](#)
- Parsează ARP reply.** Atunci când routerul primește un pachet de tip ARP reply, îl va adăuga în cache-ul ARP local. În plus, routerul va parcurge lista de pachete care așteaptă răspunsuri ARP și le va trimite pe cele pentru care adresa următorului hop este cunoscută.

Un ARP Request către router poate avea ca destinație broadcast sau adresa MAC a interfeței router-ului.

În slideshow-ul de mai jos găsiți un exemplu de utilizare a protocolului ARP pentru a determina adresa MAC a unui dispozitiv la care suntem direct conectați.

În mod normal, hostii sunt conectați la un switch, astfel ARP Request-ul este trimis la mai multe dispozitive, nu doar la router.

Protocolul ICMP

Protocolul ICMP descris în [RFC 792](#) este folosit de routere în procesul de dirijare pentru a transmite informații legate de conectivitatea la nivel IP și transport gazdelor care au trimis datagrama în cauza. Protocolul ICMP se situează deasupra IP în stiva de protocole, dar nu este un protocol de transport, ci un protocol de control (i.e. de debugging) pentru IP.

De exemplu, în anumite situații în care un pachet este aruncat de router, un mesaj ICMP este generat de router și trimis către expeditorul pachetului. În cadrul temei, ne vor interesa doar următoarele situații și mesaje:

- Destination unreachable** (type 3, code 0) - Trimis în cazul în care nu există rută până la destinație, atunci când pachetul nu este destinat routerului.
- Time exceeded** (type 11, code 0) - Trimis dacă pachetul este aruncat din cauza expirării câmpului TTL.

Pentru ambele tipuri de mesaj de eroare, pachetul emis de router trebuie să conțină, deasupra headerului ICMP, headerul de IPv4 al pachetului dropped, precum și primii 64 de biți din **payload-ul** pachetului original (adică doar ce se află deasupra antetului IPv4).

De asemenea, routerul fiind și el o entitate în rețea, poate primi mesaje ICMP de tip "Echo request" (type 8, code 0) destinate lui însuși. Acesta trebuie să răspundă cu un mesaj ICMP de tip "Echo reply" (type 0, code 0). În cazul acestor mesaje, octeții 4-5 din header capătă semnificația unui "număr de identificare", iar octeții 6-7 ai unui "număr de secvență".

Interpretarea acestora este de datoria hostului care a emis pachetele ICMP, routerul trebuie doar să se asigure că păstreze aceleași valori în pachetul de tip "Echo reply". Routerul trebuie să trimită înapoi și orice date care se aflau deasupra antetului ICMP în pachetul original.

Header-ul ICMP este următorul:

```

0                               1                               2                               3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-----+-----+-----+
| Type | Code | Checksum |
+-----+-----+-----+
| Depends on type and code |
+-----+-----+-----+

```

De exemplu, din [RFC 792 pagina 6](#), aflăm că răspunsul de tip Time Exceeded Message o să conțină la final, header-ul IP al pachetului aruncat și 64 de biți din datele cărute de IP, în acest caz un ICMP echo. Aveți mai jos o reprezentare a unui astfel de răspuns.

```

0                               1                               2                               3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-----+-----+-----+-----+-----+-----+-----+
| Type | Code | Checksum |
+-----+-----+-----+-----+-----+-----+
| unused |
+-----+-----+-----+-----+-----+-----+
| Internet Header + 64 bits of Original Data Datagram |
+-----+-----+-----+-----+-----+-----+

```

Pentru a înțelege mai bine funcționalitatea, vă recomandăm să urmăriți cu Wireshark răspunsurile routerului din casă. De exemplu, dacă rulăm ping -c 1 -t 1 8.8.8.8 de pe calculatorul nostru, vom putea observa răspunsul de tip Time Exceeded Message trimis de către router.

Vom folosi următoarea structura pentru a reprezenta header-ul ICMP:

```

struct icmp_hdr
{
    uint8_t mtype; /* message type */
    uint8_t mcode; /* type sub-code */
    uint16_t check; /* checksum */
    union
    {
        struct
        {
            uint16_t id;
            uint16_t seq;
        } echo_t; /* echo datagram. Vom folosi doar acest câmp din union */
        uint32_t gateway_addr; /* Gateway address. Nu este relevant pentru tema */
    }
}

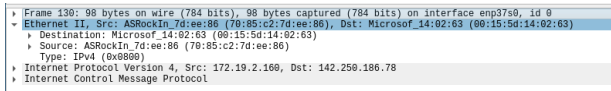
```

```

uint16_t      __unused;
uint16_t      mtu;
} frag_t;      /* Nu este relevant pentru tema */
} un_t;
};

```

O captură de pachet ICMP request în Wireshark este următoarea:



Endianness

Toate datele pe care le vom primi de pe un link sunt în Network Order, pe când procesorul de pe calculatorul nostru lucrează în Host Order. Astfel, când o să afișăm tipuri de date mai mari de un byte, va trebui să le trecem în host order cu funcții precum ntohs.

În același fel, atunci când completăm diferitele headere, va trebui să o facem în Network Order, altfel celelalte dispozitive din internet care folosesc această reprezentare le vor interpreta greșit. De exemplu, atunci când completăm câmpul ethertype din header-ul Ethernet, vom folosi eth_hdr->ether_type =

htons(ETHERTYPE_ARP); unde ETHERTYPE_ARP este 0x0806.

```
#include <arpa/inet.h>
```

```

// host to network long
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
// network to host long
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort)

```

API

Pentru rezolvarea temei vă punem la dispoziție un schelet de cod care implementeze unele funcționalități esențiale pentru rezolvarea cerințelor, precum și unele funcții ajutoare a căror utilizare este opțională. În **protocols.h** găsiți structuri pentru protocoalele cu care vom interacționa. În **lib.h** veți găsi mai mulți funcții auxiliare utile, printre care:

- **Recepționare/trimitere pachete:** aveți la dispoziție următoarele două funcții de la nivelul Datalink:

```

// include/lib.h
/* Scrie în frame_data continutul unui cadru L2 de Ethernet. În cazul nostru, frame_data
   va fi structurat astfel:

   |Alte protocoale encapsulate de IP (e.g. ICMP)|
   |IP or ARP|
   |Ethernet L2 frame| |<IP or ARP>|

   Returnează interfața pe care cadrul a fost primit. Funcția este blocantă.
*/
size_t recv_from_any_link(char *frame_data, size_t *length);

/* Trimite frame_data ca payload al unui cadru Ethernet L2.
   Link-ul pe care îl va trimite este identificat de interfață.
*/
int send_to_link(size_t length, char *frame_data, size_t interface);

```

Pentru a primi punctajul pe temă, trebuie să folosiți acest API de send/recv

- **Intrări în tabela de routare:** puteți modela o intrare în tabela de routare folosind următoarea structură:

```

/* Route table entry */
struct route_table_entry {
    uint32_t prefix;
    uint32_t next_hop;
    uint32_t mask;
    int interface;
} __attribute__((packed));

```

- **Parsare tabela de routare:** pentru a parsa tabela de routare, puteți folosi funcția:

```
int read_rtable(const char *filepath, struct route_table_entry *rtable);
```

Intrările din tabela de rutare sunt deja în network order.

- **Intrări în tabela ARP:** puteți modela o intrare în tabela ARP folosind următoarea structură:

```

struct arp_table_entry {
    uint32_t ip;
    uint8_t mac[6];
};

```

- **Parsare tabela statică ARP:** în cazul în care doriți să folosiți tabela statică de ARP, puteți să o parsați folosind funcția:

```
int parse_arp_table(char *path, struct arp_table_entry *arp_table);
```

Atentie, functia salveaza datele in **network order**.

- **Calcul sume de control:** pentru a realiza calcularea/verificarea sumelor de control din IPv4, respectiv ICMP, puteți folosi următoarea funcție:

```

/* Atunci cand calculam checksum-ul header-ului, vom pune
   campul checksum din header pe 0. */
uint16_t checksum(uint16_t *data, size_t len);

```

- **Coadă de pachete:** după cum este menționat în [descrierea protocolului ARP](#), veți avea nevoie să folosiți o coadă pentru pachete. Vă punem la dispoziție implementarea unei cozi cu elemente generice; urmăriți comentariile din include/queue.h.

- **Determinarea MAC interfață proprie:** pentru a determina adresa MAC a unei interfețe a routerului, folosiți funcția:

```
void get_interface_mac(size_t interface, uint8_t *mac);
```

Argumentul mac trebuie să indice către o zonă de memorie cu cel puțin șase octeți alocați.

- **Determinarea adresei IP:** Următoarea funcție returnează o adresă IP în format string. Pentru a trece adresa în format binar puteți folosi inet_pton.

```
char *get_interface_ip(int interface);
```

Cerințe temă

În [acest repo](#) găsiți scheletul temei, infrastructura și checker-ul automat.

Pentru rezolvarea temei, trebuie să implementați dataplane-ul unui router. **Vă recomandăm să folosiți cel puțin ping pentru a testa implementarea și Wireshark sau tcpdump pentru depanare și analiza corectitudinii.** Punctajul este împărțit în mai multe componente, după cum urmează:

- **Procesul de dirijare (30p).** Va trebui să implementați pașii prezentați în [secțiunea IPv4](#). Pentru acest exercițiu nu este nevoie să implementați și funcționalitatea referitoare la ICMP și puteți folosi o tabelă statică de ARP.
- **Longest Prefix Match eficient (16p).** La laborator, am implementat LPM folosind o căutare liniară, i.e. trecând de fiecare dată prin toate intrările din tabel. În practică, o tabelă de routare poate conține foarte multe intrări ¹ -- astfel de căutare este ineficientă. O abordare mai bună este să folosim o [trie](#).

Orice implementare mai eficientă decât căutarea liniară valorează 16 puncte.

- **Protocolul ARP (33p).** Vom implementa pașii din [secțiunea ARP](#) pentru a popula dinamic tabela ARP. Va trebui să implementați și funcționalitatea de caching; după ce primiți un răspuns ARP, rețineți adresa MAC a hostului interogată. În realitate, intrările dintr-un cache sunt temporare, fiind șterse după un anumit interval de timp. Pentru implementarea temei, este suficient să folosiți **intrări permanente**. **Pentru a nu bloca routerul, pachetele pentru care așteptăm un ARP Reply vor fi puse într-o coadă.**

Vă recomandăm să folosiți comanda arping pentru a face cereri ARP. Comanda arp afișează tabela ARP a unui host.

Tema se poate preda și cu o tabelă de ARP statică, cu pierderea punctajului aferent. În arhivă aveți inclusă o astfel de tabelă. **Prezența acestui fișier în arhivă va opri checkerul din a rula teste de ARP, deci nu veți primi puncte pe ele. Dacă ați implementat protocolul ARP, nu adăugați fișierul arp_table.txt.**

Este normal ca testul forward să pice dacă folosiți tabela statică de ARP.

- **Protocolul ICMP (21p).** Implementați pe router funcționalitatea descrisă în [secțiunea ICMP](#). Puteți folosi ping -t 1 pentru a trimite un pachet cu TTL 1. Vă recomandăm să testați și cu traceroute.

Tema trebuie să includă un README. **Pentru a fi punctat un subiect, în arhivă trebuie inclus un screenshot cu numele subiectX.png, unde X este 1, 2 sau 3, care să cuprindă unul sau mai multe terminale și cel puțin unul cu echo dat cu numele vostru, o fereastră de Wireshark cu pachetele relevante și o explicație în README a funcționalității surprinse în screenshot.**

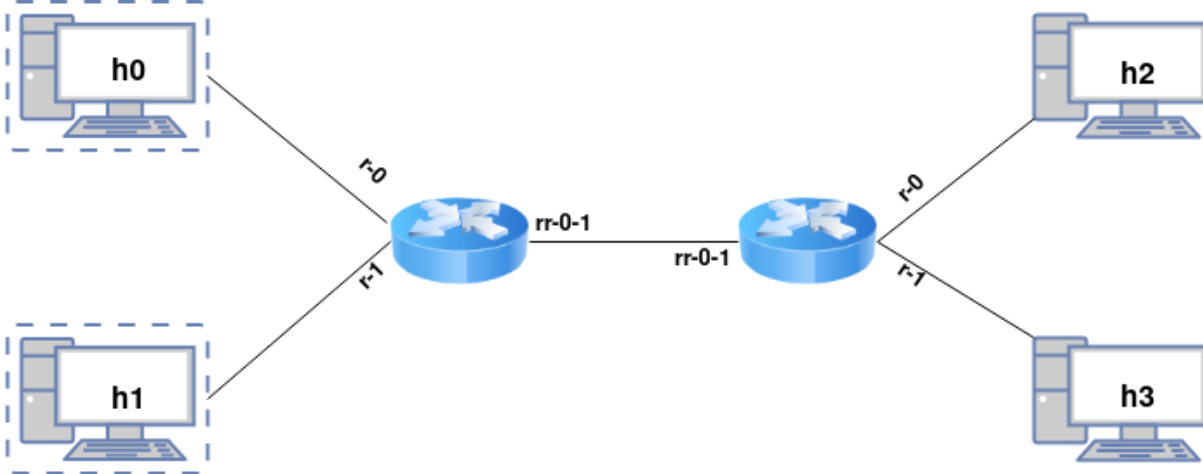
Notă: Tema poate fi implementată în C sau C++.

1

<https://blog.apnic.net/2022/01/06/bgp-in-2021-the-bgp-table/>

Testare

Vom folosi mininet pentru a simula o rețea cu următoarea topologie:



Aveți la dispoziție un script de Python3, topo.py, pe care îl puteți rula pentru a realiza set-upul de testare. Acesta trebuie rulat ca root:

```
$ sudo python3 checker/topo.py
```

Astfel, se va inițializa topologia virtuală și se va deschide câte un terminal pentru fiecare host și câte un terminal pentru fiecare router; terminalele pot fi identificate după titlu.

Fiecare host e o simplă mașină Linux, din al cărei terminal puteți rula comenzi care generează trafic IP pentru a testa funcționalitatea routerului implementat. Vă recomandăm [arping](#), [ping](#) și [netcat](#). Mai mult, din terminal putem rula wireshark sau tcpdump pentru a face inspecția de pachete.

Pentru a compila codul vom folosi make care creează binarul router.

Pentru a porni routerul manual folosim următoarele comenzi, prima pe router 0 și a doua pe router 1:

```
make run_router0 # din terminalul lui router0
make run_router1 # din terminalul lui router1
```

Ca să nu scrieți manual IP-ul unui host, puteți folosi h0, h1, h2 și h3 în loc de IP. (e.g. ping h1)

În cazul în care un pachet este aruncat (dropped) de host-ul destinație, vă recomandăm să validați următoarele lucruri folosind Wireshark pe host-ul unde pachetul este aruncat, pentru depanarea rapidă:

- Verificați că pachetul a ajuns pe interfața bună
- Verificați adresa MAC destinație
- Verificați adresa IP destinație
- Verificați checksum-ul cu Wireshark (folosind validate din Wireshark: Right Click -> Protocol Preferences -> Validate the checksum if possible)
- Verificați ca tot_length în IP header să fie corect.

Testare automată

Înainte de a folosi testele automate, vă recomandăm să folosiți modul interactiv al temei pentru a vă verifica corectitudinea implementării. Testarea automată durează câteva minute, așa că este mult mai rapid să testați manual.

De asemenea, vă punem la dispoziție și o suită de teste:

```
$ ./checker/checker.sh
```

În urma rulării testelor, va fi generat un folder host_outputs care conține, pentru fiecare test, un folder cu outputul tuturor hostilor (ce au scris la stdout și stderr). În cazul unui test picat, s-ar putea să găsiți utilă informația de aici, mai ales cea din fișierele de stderr. Folderul conține și câte un fișier pcap pentru fiecare router, pe care îl puteți inspecta apoi în Wireshark (captura este făcută pe toate interfețele routerului, deci pachetele dirijate vor apărea de două ori; urmăriți indicațiile de [aici](#) pentru a obține o vizualizare mai bună).

Puteți rula un singur test folosind argumentul run . De exemplu:

```
sudo python3 checker/topo.py run router_arp_reply
```

Nu veți primi un raport PASS/FAIL, dar veți putea obține ușor outputul din host_output.

Notă: Scopul testelor este de a ajuta cu dezvoltarea și evaluarea temei. Mai presus de rezultatul acestora, important este să implementați *cerința*. Astfel, punctajul final poate diferi de cel tentativ acordat de teste, în situațiile în care cerința temei nu este respectată (un caz extrem ar fi hardcodarea outputului pentru fiecare test în parte). Vă încurajăm să utilizați modul interactiv al topologiei pentru a explora și alte moduri de testare a temei (e.g. ping, arping).

Testele nu trimit ICMP-uri, ci doar pachete IP

Descrierea testelor automate este următoarea:

router_arp_reply - De pe h0 se trimite un ARP request către router; se verifică că routerul a trimis un reply; (practic un arping)

router_arp_request - De pe h0 se trimite un pachet către h1; ca routerul să-l poată livra, trebuie deci să trimită un ARP request către h1 - acesta e singurul verificat (nu se verifică dacă pachetul trimis de h0 chiar a ajuns pe h1)

forward - De pe h0 se trimite un pachet către h1, se verifică că a ajuns

forward_no_arp - De pe h0 se trimit, pe rând, două pachete către h1; pentru că routerul trebuie să rețină informațiile primite de la un ARP reply, se verifică că pe h1 ajunge *cel mult* un ARP request

ttl - La fel că "forward", dar se verifică și dacă TTL-ul a fost decremenat

checksum - La fel că "forward", dar se verifică și dacă checksumul e corect

wrong_checksum - De pe h0 se trimite pe h1 un pachet IP cu checksum greșit; se verifică că nu ajunge pe h1

forwardXY - Se trimite un pachet de la hX la hY; în rest, identic cu "forward"

router_icmp - De pe h0, se trimite un ICMP echo request către router; se verifică că routerul răspunde

icmp_timeout - Se trimite de pe h0 un pachet cu TTL=1; se verifică că routerul răspunde cu ICMP time-exceeded

host_unreachable - Se trimite de pe h0 un pachet către un host necunosct; se verifică că routerul răspunde cu ICMP host-unreachable

forward10packets - Se trimit 10 pachete de la h0 la h1

forward10across - Se trimit 10 pachete de la h0 la h3 (deci trebuie să treacă prin două route

[Trimitere](#)

Pentru a fi notată în catalog, tema va fi trimisă pe Moodle, unde checker-ul va fi rulat automat și va pune la feedback nota și output-ul rulării. O rulare completă durează cam 6 minute.

Se vor trimite doar arhive generate folosind scriptul create_archive.sh din schelet. Vă rugăm să trimiteți tema pe Moodle doar după ce ați verificat că aceasta rulează bine pe local.

Tema nu va modifica conținutul fisierului lib.c și include.

Checker-ul rulează peste un Ubuntu 20.04 cu următoarea versiune de GCC:

gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0 Copyright (C) 2019 Free Software Foundation, Inc. This is free software; see the source for copying conditions. There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

```

#include "protocol.h"
#include "queue.h"
#include "lib.h"
#include "string.h"
#include "arpa/inet.h"
#include "stdio.h"

// routing table
struct route_table_entry *rtable;
int rtable_len;

// ARP table
struct arp_table_entry *arp_table;
int arp_table_len = 0;

struct trie_node {
    struct trie_node *fa[2];
    struct route_table_entry *entry;
};

static struct trie_node *trie_root;

static struct trie_node *new_trie_node(void)
{
    struct trie_node *node = calloc(1, sizeof(struct trie_node));
    DIE(node == NULL, "calloc failed");
    return node;
}

static int ljunime_prefix(uint32_t mask)
{
    int ljunime = 0;
    uint32_t primul_bit = (31 << 3);
    while (mask & primul_bit) { // nu poate exista un bit 0 intre doi biti 1
        ljunime++;
        mask = mask << 1;
    }
    return ljunime;
}

static void trie_insert(struct route_table_entry *entry)
{
    uint32_t prefix = stoh(entry->prefix);
    uint32_t mask = stoh(entry->mask);
    int prefix_size = ljunime_prefix(mask);

    struct trie_node *node = trie_root;

    for (int i = 31; i >= 31 - prefix_size; i--) {
        int bit = (prefix >> i) & 1;
        if (node->fa[bit] == NULL) // nu exista nod pt bit
            node->fa[bit] = new_trie_node();
        node = node->fa[bit];
    }

    if (node->entry == NULL || stoh(entry->mask) > stoh(node->entry->mask)) // masca mai specifica sau primul entry pentru prefix
        node->entry = entry;
}

// lista cu trie
struct route_table_entry *get_best_route(uint32_t target_ip)
{
    uint32_t ip = stoh(target_ip);
    struct trie_node *node = trie_root;
    struct route_table_entry *best = NULL;

    for (int i = 31; i >= 0; i--) {
        if (node->entry != NULL)
            best = node->entry;

        int bit = (ip >> i) & 1;
        if (node->fa[bit] == NULL)
            break;
        node = node->fa[bit];
    }

    if (node != NULL && node->entry != NULL) // mai specific decat best
        best = node->entry;

    return best;
}

// get_mac_entry din tabela ARP
struct arp_table_entry *get_mac_entry(uint32_t given_ip)
{
    for (int i = 0; i < arp_table_len; i++) {
        if (arp_table[i].ip == given_ip)
            return arp_table[i];
    }
    return NULL;
}

```

```

// request ARP pentru o adresa mac si asociat un IP
void send_arp_request(uint32_t target_ip, int out_interface)
{
    char packet[sizeof(struct ether_hdr) + sizeof(struct arp_hdr)]; // 14 + 28 = 42 bytes
    memset(packet, 0, sizeof(packet));

    struct ether_hdr *eth_packet = (struct ether_hdr *)packet; // layer 2
    struct arp_hdr *arp_packet = (struct arp_hdr *)packet + sizeof(struct ether_hdr); // layer 3

    uint8_t mac[6];
    get_interface_mac(out_interface, mac);
    int_pton(AF_INET, get_interface_ip(out_interface), &ip); // AF_INET convertește string IP in uint32_t in rețea

    memset(eth_packet->ether_dhost, 0xFF, 6); // broadcast
    memcpy(eth_packet->ether_shost, mac, 6);
    eth_packet->ether_type = htons(0x0806);

    arp_packet->hw_type = htons(1); // 1 = Ethernet
    arp_packet->proto_type = htons(0x0800);
    arp_packet->hw_len = 6;
    arp_packet->proto_len = 4;
    memcpy(arp_packet->opcode, &htons(1)); // 1 = request
    memcpy(arp_packet->sender_mac, mac, 6);
    arp_packet->sender_ip = ip;
    memset(arp_packet->target_mac, 0, 6); // MAC dest necunoscut
    arp_packet->target_ip = target_ip;

    send_to_link(sizeof(packet), packet, out_interface);
}

void send_arp_reply(struct arp_hdr *req, int in_interface)
{
    char packet[sizeof(struct ether_hdr) + sizeof(struct arp_hdr)]; // 14 + 28 = 42 bytes
    memset(packet, 0, sizeof(packet));

    struct ether_hdr *eth_packet = (struct ether_hdr *)packet; // layer 2
    struct arp_hdr *arp_packet = (struct arp_hdr *)packet + sizeof(struct ether_hdr); // layer 3

    uint8_t my_mac[6];
    get_interface_mac(in_interface, my_mac);
    int_pton(AF_INET, get_interface_ip(in_interface), &arp_packet->proto_ip); // AF_INET convertește string IP in uint32_t in rețea

    memcpy(eth_packet->ether_dhost, req->ether_dhost, 6); // MAC sursa din request
    memcpy(eth_packet->ether_shost, my_mac, 6);
    eth_packet->ether_type = htons(0x0806);

    arp_packet->hw_type = req->hw_type;
    arp_packet->proto_type = req->proto_type;
    arp_packet->hw_len = req->hw_len;
    arp_packet->proto_len = req->proto_len;
    memcpy(arp_packet->opcode, &htons(2)); // 2 = reply
    memcpy(arp_packet->sender_mac, req->sender_mac, 6);
    memcpy(arp_packet->sender_ip, req->sender_ip, 4);
    memcpy(arp_packet->target_mac, req->target_mac, 6);
    memcpy(arp_packet->target_ip, req->target_ip, 4);

    send_to_link(sizeof(packet), packet, in_interface);
}

// time exceeded sau destination unreachable
void send_icmp_error(uint8_t tip, uint8_t cod, char *buf_original, size_t len_original, int in_interface)
{
    struct ether_hdr *eth_original = (struct ether_hdr *)buf_original;
    struct ip_hdr *ip_original = (struct ip_hdr *)buf_original + sizeof(struct ether_hdr); // layer 3

    int ljunime_hdr_ip_original = ip_original->ihl * 4; // ljunimea headerului ip original
    int ljunime_icmp = ljunime_hdr_ip_original + 8; // din moment ce la biti din payload al pachetului original
    size_t ljunime_totala = sizeof(struct ether_hdr) + sizeof(struct ip_hdr) + sizeof(struct icmp_hdr) + ljunime_icmp;
    char *raspuns = calloc(1, ljunime_totala);
    if (raspuns == NULL)
        return;

    struct ether_hdr *eth_nou = (struct ether_hdr *)raspuns;
    memcpy(eth_nou->ether_dhost, eth_original->ether_dhost, 6);
    get_interface_mac(in_interface, eth_nou->ether_shost);
    eth_nou->ether_type = htons(0x0800); // IPv4

    struct ip_hdr *ip_nou = (struct ip_hdr *)raspuns + sizeof(struct ether_hdr); // layer 3
    ip_nou->version = 4;
    ip_nou->frag = 0;
    ip_nou->ttl = 64;
    ip_nou->tot_len = htons((uint16_t)(sizeof(struct ip_hdr) + sizeof(struct icmp_hdr) + ljunime_icmp));
    ip_nou->ttl = 64;
    ip_nou->proto = 1; // ICMP
    ip_nou->checksum = 0;
    int_pton(AF_INET, get_interface_ip(in_interface), &ip_nou->source_addr);
    ip_nou->dest_addr = ip_original->source_addr;
    ip_nou->checksum = htons(checksum((uint16_t *)ip_nou, sizeof(struct ip_hdr)));

    struct icmp_hdr *icmp_nou = (struct icmp_hdr *)raspuns + sizeof(struct ether_hdr) + sizeof(struct ip_hdr); // layer 4
    icmp_nou->type = tip;
    icmp_nou->code = cod;
    icmp_nou->checksum = 0;
    icmp_nou->seq = 0;
    icmp_nou->echo_id = 0;
    icmp_nou->echo_seq = 0;
    uint8_t *data_icmp = (uint8_t *)icmp_nou + sizeof(struct icmp_hdr);
    memcpy(data_icmp, ip_original->payload, ljunime_icmp);
    icmp_nou->checksum = htons(checksum((uint16_t *)icmp_nou, sizeof(struct icmp_hdr) + ljunime_icmp));

    send_to_link(ljunime_totala, raspuns, in_interface);
}

```

```

void send_icmp_echo_reply(char *buf, size_t len, int in_interface)
{
    struct ether_hdr *eth = (struct ether_hdr *)buf;
    struct ip_hdr *ip = (struct ip_hdr *)buf + sizeof(struct ether_hdr);
    struct icmp_hdr *icmp = (struct icmp_hdr *)buf + sizeof(struct ether_hdr) + ip->ihl * 4;

    uint8_t mac[6];
    memcpy(mac, eth->ether_dhost, 6);
    memcpy(eth->ether_dhost, eth->ether_shost, 6);
    memcpy(eth->ether_shost, mac, 6);

    uint32_t ip_nou = ip->dest_addr;
    ip->dest_addr = ip->source_addr;
    ip->source_addr = ip_nou;

    ip->ttl = 64;
    ip->checksum = 0;
    ip->checksum = htons(checksum((uint16_t *)ip, ip->ihl * 4));
    icmp->type = 0;
    icmp->code = 0;
    icmp->checksum = 0;
    int ljunime_icmp = stoh(ip->tot_len) - ip->ihl * 4;
    icmp->checksum = htons(checksum((uint16_t *)icmp, ljunime_icmp));

    send_to_link(len, buf, in_interface);
}

int main(int argc, char *argv[])
{
    char buf[MAX_PACKET_LEN];

    // Do not modify this line
    Init(argc > 2, argv > 2);

    rtable = malloc(sizeof(struct route_table_entry) * 100000);
    DIE(rtable == NULL, "memory");

    rtable_len = read_rtable(argv[1], rtable);
    DIE(rtable_len < 0, "read_rtable");

    trie_root = new_trie_node();
    for (int i = 0; i < rtable_len; i++)
        trie_insert(&rtable[i]);

    queue_coada_de_pachete = create_queue();

    while (1) {
        size_t interface;
        size_t len;

        interface = recv_from_any_link(buf, &len);
        DIE(interface < 0, "recv_from_any_links");

        // TODO: Implement the router forwarding logic

        /* Note that packets received are in network order,
        any header field which has more than 1 byte will need to be converted to
        host order. For example, stoh(eth_hdr->ether_type). The opposite is needed when
        sending a packet on the link. */

        struct ether_hdr *eth_hdr = (struct ether_hdr *)buf;

        uint8_t mac[6];
        uint8_t broadcast[6] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
        get_interface_mac(interface, mac);

        // din enunt: routerul nostru trebuie sa considere doar pachetele trimise catre el inusi sau catre toata lumea (FF:FF:FF:FF:FF:FF)
        if (memcmp(eth_hdr->ether_dhost, mac, 6) != 0 && memcmp(eth_hdr->ether_dhost, broadcast, 6) != 0)
            continue;

        // Din (enunt) iar ARP (enunt)
        uint16_t ether_type = stoh(eth_hdr->ether_type);

        // ARP
        if (ether_type == 0x0806) {
            if (len < sizeof(struct ether_hdr) + sizeof(struct arp_hdr))
                continue;

            struct arp_hdr *arp_hdr = (struct arp_hdr *)buf + sizeof(struct ether_hdr);
            uint16_t operate = stoh(arp_hdr->opcode);

            if (operate == 1) { // request
                uint32_t ip;
                int_pton(AF_INET, get_interface_ip(interface), &ip);
                if (arp_hdr->proto == ip)
                    send_arp_reply(arp_hdr, interface);
            }
            else if (operate == 2) { // reply
                if (get_arp_entry(arp_hdr->proto) == NULL) {
                    arp_table = realloc(arp_table, (arp_table_len + 1) * sizeof(struct arp_table_entry));
                    DIE(arp_table == NULL, "realloc failed");
                    arp_table[arp_table_len].ip = arp_hdr->proto;
                    memcpy(arp_table[arp_table_len].mac, arp_hdr->sender_mac, 6);
                    arp_table_len++;
                }
                queue_coada = create_queue();
                while (!queue_empty(coada_de_pachete)) {
                    char *packet = queue_deq(coada_de_pachete);

                    uint32_t *ip_packet = (uint32_t *)packet;
                    int *interface_packet = (int *)packet + sizeof(uint32_t);
                    size_t *len_packet = (size_t *)packet + sizeof(uint32_t) + sizeof(int);
                    char *buf_packet = packet + sizeof(uint32_t) + sizeof(int) + sizeof(size_t);

                    if (*ip_packet == arp_hdr->proto) { // este pachetul nostru ARP pentru IP al din reply
                        struct ether_hdr *eth_resend = (struct ether_hdr *)buf_packet;
                        get_interface_mac(*interface_packet, eth_resend->ether_dhost);
                        memcpy(eth_resend->ether_shost, arp_hdr->sender_mac, 6);

                        send_to_link(*len_packet, buf_packet, *interface_packet);
                    }
                    else { // este un request ARP pentru alt
                        queue_enqueue(coada, packet);
                    }
                }

                while (!queue_empty(coada)) { // pachetele care asteapta ARP reply pentru alte IP-uri
                    queue_enqueue(coada_de_pachete, queue_deq(coada));
                }
            }
            continue;
        }

        if (ether_type != 0x0800) // IPv4
            continue;

        // ether_hdr = 6 MACs + 6 MACs + 2 b type = 14 bytes
        // ip_hdr = 28 bytes
        if (len < sizeof(struct ether_hdr) + sizeof(struct ip_hdr)) // packet incomplet/malformat
            continue;

        struct ip_hdr *ip_hdr = (struct ip_hdr *)buf + sizeof(struct ether_hdr); // layer 3

        uint16_t old_checksum = ip_hdr->checksum;
        ip_hdr->checksum = 0;
        uint16_t new_checksum = checksum((uint16_t *)ip_hdr, ip_hdr->ihl * 4);
        ip_hdr->checksum = old_checksum;

        if (new_checksum != old_checksum)
            continue;

        int destinet_routerul = 0;
        for (int i = 0; i < argc - 2; i++) {
            uint32_t ip_interface;
            int_pton(AF_INET, get_interface_ip(i), &ip_interface);
            if (ip_hdr->dest_addr == ip_interface) {
                destinet_routerul = 1;
                break;
            }
        }

        if (destinet_routerul) {
            if (ip_hdr->proto == 1) { // raspunde doar la ICMP echo request (ping)
                struct icmp_hdr *icmp_hdr = (struct icmp_hdr *)buf + sizeof(struct ether_hdr) + ip_hdr->ihl * 4; // layer 4
                if (icmp_hdr->type == 0 && icmp_hdr->code == 0)
                    send_icmp_echo_reply(buf, len, interface);
            }
            continue;
        }

        if (ip_hdr->ttl <= 1) {
            send_icmp_error(11, 0, buf, len, interface); // time exceeded
            continue;
        }

        struct route_table_entry *best_route = get_best_route(ip_hdr->dest_addr);
        if (best_route == NULL) {
            send_icmp_error(3, 0, buf, len, interface); // ICMP destination unreachable
            continue;
        }

        ip_hdr->ttl--;
        ip_hdr->checksum = 0;
        ip_hdr->checksum = htons(checksum((uint16_t *)ip_hdr, ip_hdr->ihl * 4));

        struct arp_table_entry *a_entry = get_arp_entry(best_route->next_hop);

        if (a_entry != NULL) { // MAC cunoscut
            get_interface_mac(best_route->interface, eth_hdr->ether_dhost);
            memcpy(eth_hdr->ether_dhost, a_entry->mac, 6);
            send_to_link(len, buf, best_route->interface);
        }
        else { // nu cunoscut
            char *packet_coada = malloc(sizeof(uint32_t) + sizeof(int) + sizeof(size_t) + MAX_PACKET_LEN);
            DIE(packet_coada == NULL, "malloc packet MAC necunoscut");

            uint32_t *target_ip = (uint32_t *)packet_coada;
            int *target_interface = (int *)packet_coada + sizeof(uint32_t);
            size_t *target_len = (size_t *)packet_coada + sizeof(uint32_t) + sizeof(int);
            char *target_buf = packet_coada + sizeof(uint32_t) + sizeof(int) + sizeof(size_t);

            *target_ip = best_route->next_hop;
            *target_interface = best_route->interface;
            *target_len = len;
            memcpy(target_buf, buf, len);

            queue_enqueue(coada_de_pachete, packet_coada);
        }

        send_arp_request(best_route->next_hop, best_route->interface);
    }
    return 0;
}

```